Université Paris Cité– UFR d'Informatique
L3 Informatique et Double Licences

Grammaires et analyse syntaxique
Année 2024-2025

Projet

Un interpréteur pour l'édition de texte en HTML

Version du 31 mars 2025
Date limite : 17 mai 2025

1 Consignes

Le projet doit être rendu avant le **17 mai 2025, à 23h59**. Le projet est à faire en binôme (pas de trinôme !). Les deux membres d'un binôme peuvent être dans des groupes de TP différents. Les monômes sont autorisés mais seront notés comme des binômes.

Rendu demandé. Le langage de programmation utilisé doit être OCaml. L'analyse lexicale doit être réalisée avec *ocamllex*, et l'analyse grammaticale avec *menhir*. Il est conseillé de vous servir des raccourcis de la bibliothèque standard de menhir. Pour la compilation, vous êtes libres de vous servir de `dune`, `ocamlbuild`, ou d'écrire un Makefile à la main.
Il n'est pas demandé de rédiger un rapport de projet ; cependant, on demande d'inclure dans le dépôt git de votre projet un fichier `ReadMe.txt` qui indique les commandes à lancer pour compiler et exécuter votre programme. Il est aussi conseillé d'inclure un sous-dossier `exemples` contenant un ou plusieurs fichiers à tester pour montrer les différentes fonctionnalités que vous aurez implémentées. De plus, il vous est demandé d'expliquer brièvement (dans le fichier `ReadMe.txt`) quelles sont les parties du sujet que vous avez réalisées, et comment vous avez réparti le travail dans le binôme.
Le projet est divisé en quatre étapes. Les étapes 1, 2 et 3 constituent la partie obligatoire du projet. L'étape 4 est facultative et laisse davantage de liberté : si vous décidez d'aller plus loin que les étapes 1-3, conservez une version stable du projet à la fin de l'étape 3 (par exemple dans une branche git) avant de vous lancer dans des expérimentations fantaisistes.
Le thème du cours étant l'analyse syntaxique, on accordera une attention toute particulière à la partie lexer/parser : pas de conflits dans la grammaire, messages d'erreurs clairs en cas de problème de syntaxe dans le fichier d'entrée.

Dépôt git. Le travail doit **obligatoirement** être réalisé via un dépôt git pour chaque groupe (binôme ou monôme). L'usage de ce gestionnaire de version permet d'éviter toute perte de donnée, et d'accéder si besoin à vos anciennes versions.

- Votre dépôt git doit être hébergé sur le serveur GitLab de l'UFR d'informatique : https://moule.informatique.univ-paris-diderot.fr
- **Votre dépôt doit être rendu privé** dès sa création, avec accès uniquement aux membres du groupe. Tout code laissé en accès libre sur moule ou ailleurs sera considéré comme une incitation à la fraude, et sanctionné. Une fois votre dépôt git créé, ajoutez **tous** les enseignants du cours, en leur assignant le rôle Maintainer : `@barszeza`, `@fagnot`, `@habermeh`, `@jledent`, `@picantin`, `@treinen`.

— Prenez soin de faire des commits réguliers (au *minimum* un pour la version basique, puis un pour chaque extension rajoutée). Faites en sorte que les messages des commits soient clairs : nouvelle fonctionnalité, correction d'un bug, refactorisation du code, ...

— Il est fortement conseillé de travailler dans une nouvelle branche lorsque vous implémentez une extension : cela vous permettra de conserver une version principale (qui marche), tout en travaillant sur une version étendue (possiblement bugguée). Évidemment, l'objectif final est idéalement d'atteindre une version finale dans laquelle toutes les extensions sont fonctionnelles.

Plagiat, code généré par IA. Il va de soi que votre travail doit être strictement personnel : aucune communication de code ou d'"idées" entre les groupes, aucune "aide" externe ou entre groupes. Nous vous rappelons que la fraude à un projet est aussi une fraude à un examen, passible de sanctions disciplinaires. L'utilisation de code généré par IA (ChatGPT, Copilot, etc.) est **interdite** et sera sanctionnée au même titre que le plagiat.

En cas de difficultés. Pour toute question concernant le projet, une section « Questions Projet » est disponible sur Moodle (évidemment, n'y postez pas de bout de code). Vous pouvez aussi poser des questions directement à votre chargé·e de TD, ou par mail en ajoutant le tag "[GAS6]" dans l'objet du mail. Veuillez signaler tout problème technique (problèmes d'installation, de compilation) **dès que possible** ! (C'est-à-dire, pas la veille du rendu et encore moins le jour de la soutenance.)

2 But du projet

L'objectif du projet est d'implémenter un langage de mise en forme de texte (inspiré de Markdown et LaTeX), appelé SimpleText. Votre travail sera d'écrire un analyseur lexical et syntaxique capable de lire des documents au format SimpleText, et de produire un document HTML équivalent.

Dans un premier temps, un document SimpleText pourra contenir les quatre éléments suivants :

— des titres, indiqués par la présence du symbole `"#"` en début de ligne ;

— des sous-titres, indiqués par un double symbole `"##"` en début de ligne ;

— des paragraphes, c'est-à-dire un bloc de texte ;

— des listes à puce, où chaque item de la liste commencera par la chaîne `"\item"`.

Les retours à la ligne simples (`"\n"`) présents dans le document source sont ignorés. Pour délimiter les différents paragraphes, on utilise un saut de ligne (`"\n\n"`). Le texte présent dans les titres, les paragraphes, et les items des listes à puce, peut être mis en forme de l'une des trois manières suivantes :

— texte en italique : on met un symbole `"*"` avant et après le texte en italique ;

— texte en gras : on met un double symbole `"**"` avant et après le texte en gras ;

— lien hypertexte : on écrit le texte du lien entre crochets `"["` et `"]"`, suivi par l'URL du lien entre parenthèses `"("` et `")"`.

Un exemple de fichier SimpleText est donné ci-dessous :

Listing 1 – exemple.stxt

```
# Exemple de fichier SimpleText

N'oubliez pas :
dans un fichier SimpleText,
les retours à la ligne
sont ignorés !

Pour faire un nouveau paragraphe, il faut faire un *saut* de ligne.
Pour mettre en forme le texte, on peut utiliser:

\item du *texte en italique*,
\item du **texte en gras**,
\item et des [liens hypertexte](https://fr.wikipedia.org/wiki/Hyperlien).
```

Une fois mis en forme, le fichier `exemple.stxt` devra donner le rendu suivant :

Exemple de fichier SimpleText

N'oubliez pas : dans un fichier SimpleText, les retours à la ligne sont ignorés !

Pour faire un nouveau paragraphe, il faut faire un *saut* de ligne. Pour mettre en forme le texte, on peut utiliser :

— du *texte en italique*,

— du **texte en gras**,

— et des liens hypertexte.

Rappels de HTML. Ce projet ne demande pas de connaissances approfondies en HTML. Si vous avez quelques lacunes, renseignez-vous sur les balises suivantes : `<h1>`, `<h2>`, `<p>`, `<ul>`, `<em>`, `<strong>`, et `<a href="URL">`.

3 Étape 1 : version basique

La grammaire du langage SimpleText telle que décrite dans la Section 2 est donnée sous forme Backus–Naur (voir Cours 3) dans la Figure 1. Dans la grammaire, conformément à la notation BNF, les non-terminaux sont écrits entre chevrons $< >$, et les terminaux sont des chaînes de caractères entre guillemets. La chaîne `"\n\n"` désigne deux retours à la ligne successifs (c'est-à-dire, un saut de ligne). Le symbole EOF est aussi un terminal, qui désigne la fin du fichier.

1. Écrire un analyseur lexical (lexer) et un analyseur syntaxique (parser) qui permettront d'extraire la structure d'un document SimpleText comme indiqué dans la Figure 1. Le lexer devra être écrit avec ocamllex, et le parser avec Menhir. Il faudra notamment déterminer les lexèmes (tokens) adéquats, ainsi que définir un type OCaml pour l'arbre de syntaxe abstraite (AST) qui représente un document SimpleText.

La syntaxe du langage SimpleText est donnée par la grammaire BNF suivante :

```
<document>        ::= <corps> EOF
<corps>           ::= <element> "\n\n" <corps> | <element>
<element>         ::= "#" <texte>
                    | "##" <texte>
                    | <texte>
                    | <suite_items>
<suite_items>     ::= <item> <suite_items> | <item>
<item>            ::= "\item" <texte>
<texte>           ::= <element_de_texte> <texte> | ε
<element_de_texte> ::= <mot>
                    | "*" <liste_mots> "*"
                    | "**" <liste_mots> "**"
                    | "[" <liste_mots> "](" <mot> ")"
<liste_mots>      ::= <mot> <liste_mots> | ε
```

où `<mot>` désigne une suite de lettres, chiffres ou symboles, sans espaces ni retours à la ligne, et où les symboles clés du langage SimpleText (*, #, \, crochets et parenthèses) sont interdits. Ainsi, un `<mot>` est donné par l'expression rationnelle suivante :

```
<mot>  ::=  [^ '*' '#' '\\' '[' ']' '(' ')' ' ' '\n' '\t']+
```

Figure 1 – La syntaxe de SimpleText

2. Ensuite, écrire un interpréteur pour l'arbre obtenu qui va produire la traduction vers un document HTML. Pour vérifier que le rendu est correct, vous pouvez ouvrir le fichier HTML obtenu dans votre navigateur web.

⚠ Avant de passer à la suite, assurez-vous que tout fonctionne pour l'étape 1 : pas de conflits dans la grammaire, erreurs correctement gérées, rendu HTML conforme.

4 Étape 2 : premières extensions simples

Nous allons maintenant étendre les possibilités du langage SimpleText. Les extensions ci-dessous sont relativement indépendantes et peuvent être implémentées dans l'ordre de votre choix. Si vous n'arrivez pas à implémenter l'une d'entre elles, cela ne vous empêche pas de passer à la partie suivante.

4.1 Meilleure détection des paragraphes

Dans la grammaire de la Figure 1, la séparation entre deux paragraphes se fait lorsqu'on rencontre deux retours à la ligne successifs (`"\n\n"`). C'est plutôt rigide :

— si on insère des espaces ou des tabulations (`"\t"`) entre les deux `"\n"`, le saut de ligne n'est pas détecté ;

— au contraire, si on saute plus de deux lignes, l'analyseur syntaxique va produire un ou plusieurs paragraphes « vides », ce qui est inutile.

On voudrait donc assouplir la syntaxe permettant de définir un nouveau paragraphe.

3. Modifier votre analyseur lexical de sorte que le token correspondant au saut de ligne soit produit lorsqu'on rencontre une suite de ≥ 2 symboles `"\n"`, éventuellement séparés par des espaces et des tabulations.

4.2 Texte en couleur

En plus du texte en gras et en italique, on voudrait pouvoir écrire du texte en couleurs. Pour cela, on utilisera la syntaxe suivante : `"\color{"` `<code_couleur>` `"}{"` `<liste_mots>` `"}"`. Le code couleur sera composé de 6 caractères hexadécimaux. Par exemple,

```
Du texte \color{FF0000}{en rouge} et \color{0000FF}{en bleu}.
```

devra donner comme rendu :

Du texte en rouge et en bleu.

---

4. Ajouter la possibilité de mettre du texte en couleur. Pour le rendu HTML, vous pourrez utiliser la balise `<span>`.

4.3 Items imbriqués

Dans la grammaire de la Figure 1, un `<item>` ne peut contenir qu'un bloc de texte. On voudrait maintenant autoriser qu'un item puisse contenir plusieurs paragraphes, ou bien une sous-liste d'items. Pour éviter que la grammaire ne soit ambiguë, il nous faut un délimiteur pour indiquer ce qui se trouve dans l'item ou pas. On utilisera les accolades `"{"` et `"}"`.

```
\item Un item simple.
\item {
  Et un item complexe.

  Il contient:

  \item des paragraphes,
  \item et des sous-items.
}
\item Un autre item simple.
```

- Un item simple
- Et un item complexe.

  Il contient :

  - des paragraphes,
  - et des sous-items.
- Un autre item simple.

---

5. Modifier la grammaire du langage SimpleText (c'est-à-dire, modifier votre lexer et/ou parser) de façon à autoriser les paragraphes dans les items et les items imbriqués. Attention à ne pas introduire de conflit !
6. Il faudra aussi modifier votre AST et le générateur de fichier HTML en conséquence.

4.4 Combiner italique, gras, liens et couleurs

La grammaire de la Figure 1 n'autorise pas d'écrire du texte à la fois en gras et en italique. On ne peut pas non plus mettre le texte d'un lien hypertexte en gras ou en italique.

7. Modifier la grammaire de sorte que l'on puisse combiner le texte en gras, en italique, les liens hypertexte, voire les couleurs si vous avez implémenté l'extension 4.2. Attention aux conflits ! Il ne suffit pas de remplacer le non-terminal `<liste_mots>` par `<texte>`.

5 Étape 3 : ajout de macros

Dans cette troisième partie, nous allons implémenter une extension un peu plus conséquente. L'objectif est de permettre l'utilisation de *macros*, c'est-à-dire, permettre à l'utilisateur de définir ses propres abbréviations qui seront remplacées par un morceau de texte choisi dans le fichier HTML généré. Pour cela, nous allons procéder aux modifications suivantes :

— le *corps* d'un fichier SimpleText (qui contient le texte à produire) sera désormais entouré de balises `"\begindocument"` et `"\enddocument"` ;

— l'*en-tête* du fichier SimpleText est la partie située avant le mot-clé `"\begindocument"` : c'est là que seront définies les macros ;

— tout ce qui se trouve après le mot-clé `"\enddocument"` est ignoré.

L'en-tête du document sera donc constituée d'une suite de définitions de macros. Une définition de macro est générée par la grammaire BNF suivante :

```
<macro>      ::=  "\define{" <identifiant> "}{" <texte> "}"
<identifiant>  ::=  "\" <nom>
```

où le `<nom>` est une suite de caractères alphabétiques :

```
<nom>   ::=  [ 'a'-'z' 'A'-'Z' ]+
```

Une fois qu'on a défini une macro, on peut utiliser son identifiant dans le corps du document ; celui-ci sera ensuite remplacé par le texte associé lors de la génération de code HTML.

Par exemple, le fichier SimpleText ci-dessous :

```
\define{\gas}{Grammaires et Analyse Syntaxique}
\define{\BleuCanard}{048B9A}
\begindocument
## Compte rendu du projet de \gas

\item Nous avons implémenté des *macros*,
\item et de \color{\BleuCanard}{jolies couleurs} !
\enddocument
```

devra donner comme rendu :

Compte rendu du projet de Grammaires et Analyse Syntaxique

— Nous avons implémenté des *macros*,

— et de jolies couleurs !

---

8. Implémenter la gestion des macros comme décrit ci-dessus. Il faudra notamment choisir une structure de données pour stocker les identifiants des macros avec le texte qui leur est associé. Vous pouvez jeter un œil aux modules `Hashtbl` ou `Map`. Sinon, une simple liste d'association fera l'affaire.
9. Veillez aussi à gérer proprement les messages d'erreur. Par exemple, si l'utilisateur essaie d'utiliser une macro qui n'a pas été définie ; ou alors s'il essaie de définir deux macros avec le même identifiant.

6 Étape 4 : extensions facultatives

S'il vous reste du temps et si le projet vous amuse, vous pouvez implémenter d'autres extensions. Cette partie du projet est optionnelle. Nous proposons ci-dessous quelques suggestions d'extensions à rajouter ; libre à vous d'en modifier la syntaxe, ou d'implémenter quelque chose de totalement différent. N'oubliez pas de documenter les choix que vous ferez (dans votre fichier Readme), et de fournir des fichiers d'exemple à tester.

Répétition de texte. Ajouter la possibilité de répéter du texte grâce à des boucles. Par exemple, on pourrait imaginer :

```
\begindocument
\for \i in range(1, 100) {
  \i kilomètres à pieds, ça use, ça use...
}
\enddocument
```

ou encore :

```
\begindocument
Liste des participants au projet:

\for \etudiant in {Diego, Amina, Liam, Kelya, Bogdan} {
  \item \etudiant
}
\enddocument
```

Conditionnelles. Ajouter la possibilité de définir des constantes booléennes dans l'en-tête du fichier, qui pourront être testées pour afficher ou non certains blocs de texte. Par exemple :

```
\boolean{\versionlongue}{false}
\begindocument
Cette phrase apparaît dans toutes les versions.

\if \versionlongue {
  Par contre, cette phrase-là n'apparaît que dans la version longue.
}
\enddocument
```

Macros paramétrées. Ajouter la possibilité de définir des macros qui peuvent prendre en entrée des paramètres. Inspirez-vous de la syntaxe de `\newcommand` en LaTeX, si vous connaissez. Par exemple :

```
\define{\veryImportant}[1]{---> **\color{FF0000}{#1}** <---}

\begindocument
  Le projet doit être rendu \veryImportant{avant} la date limite !
\enddocument
```